AWS IAM & EC2 Security Access Control Project Report

Name: Mohammad Athar Shareef

Program: Cybersecurity

GitHub Repository: https://github.com/atharshareef7/aws-iam-ec2-security-project

1. Executive Summary

This project explores the design and implementation of AWS Identity and Access Management (IAM) and Elastic Compute Cloud (EC2) to establish a secure, role-based access control mechanism within a cloud infrastructure. The goal was to create separate production and development environments and ensure that access was tightly controlled through IAM policies and environment tagging. This setup simulates a real-world DevOps and cybersecurity scenario, demonstrating how operational flexibility and security compliance can be balanced in a modern cloud environment. By combining AWS IAM policies, EC2 instances, and tagging strategies, the project illustrates the importance of fine-grained permissions and controlled access boundaries.

2. Objectives

- Launch EC2 instances for production and development environments.
- Configure environment-based tagging for resource identification (Env=production / Env=development).
- Create IAM policies restricting access based on resource tags.
- Apply conditional statements in IAM policies using ec2:ResourceTag/Env.
- Set up IAM users and groups with appropriate permissions.
- Assign IAM policies to groups for easier permission management.
- Implement tag-based access control (Attribute-Based Access Control — ABAC).
- Prevent privilege escalation by denying tag creation and deletion actions.
- Test IAM permissions for both production and development instances.
- Validate permissions using the IAM Policy Simulator.
- Demonstrate policy enforcement through AWS Management Console.
- Apply the Principle of Least Privilege across all IAM users.
- Ensure environment isolation between development and production.
- Follow AWS security best practices and governance standards.
- Simulate a real-world DevSecOps access control scenario
- Implement role-based access control (RBAC) for interns and developers.
- Demonstrate secure login via AWS Account Alias.

- Manage user access through centralized IAM groups.
- Monitor and verify policy outcomes through audit-friendly actions.
- Document the entire process with screenshots and policy references.

3. Tools and Services Used

Tool / Service	Purpose
AWS EC2	Launch and manage virtual machines.
AWS IAM	Manage users, groups, and permissions.
AWS Management Console	Graphical interface for AWS operations.
IAM Policy JSON	Defines access control rules.
AWS Account Alias	Simplifies IAM user login process.

4. Implementation Steps

Step 1: Created an Account Alias for easier IAM user login.

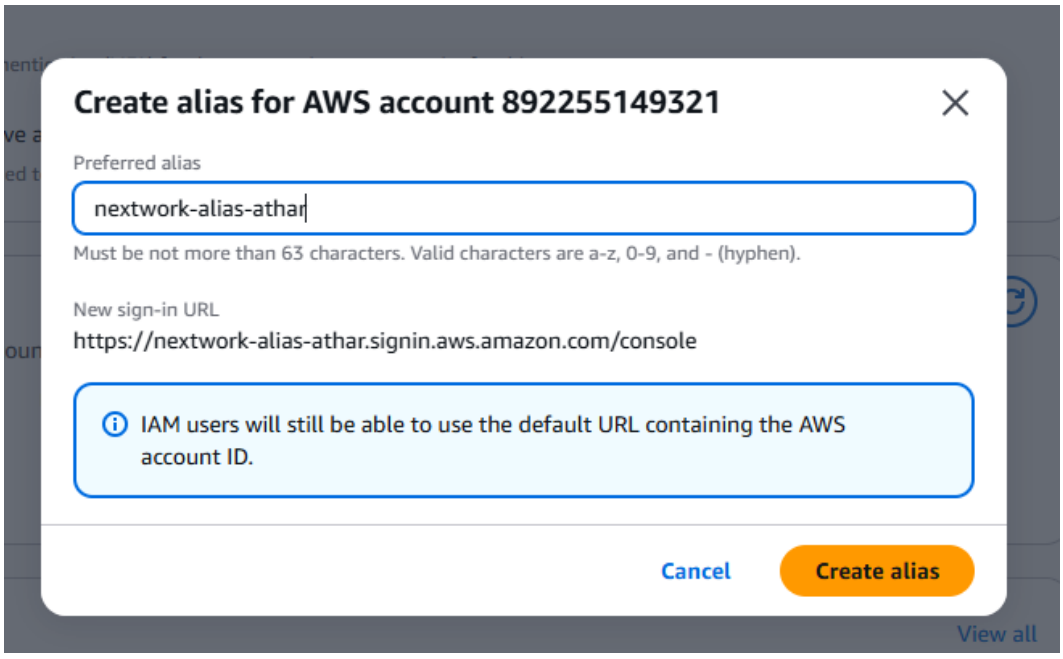

Figure 1: AWS Account Alias Creation (nextwork-alias-athar)

Step 2: Created EC2 instances for production and development environments with respective tags.

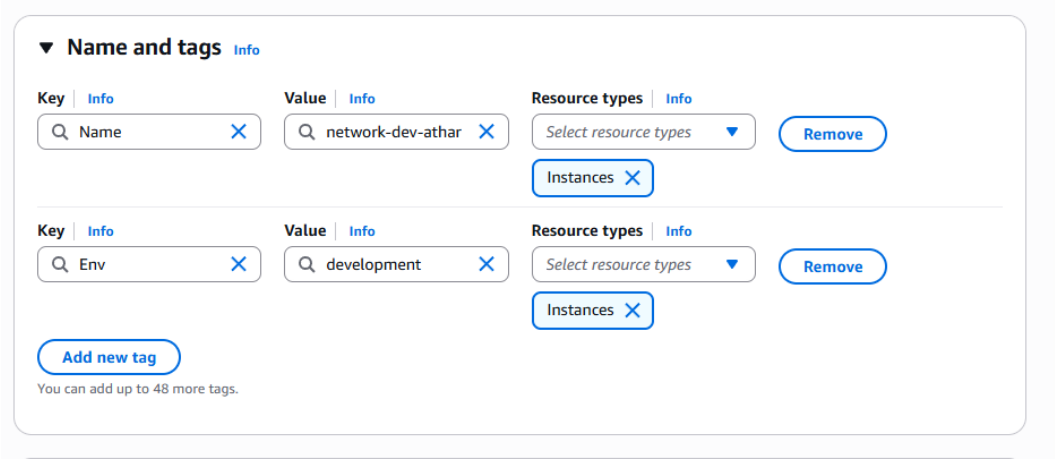

Figure 2: Development Instance Tagged with Env=development

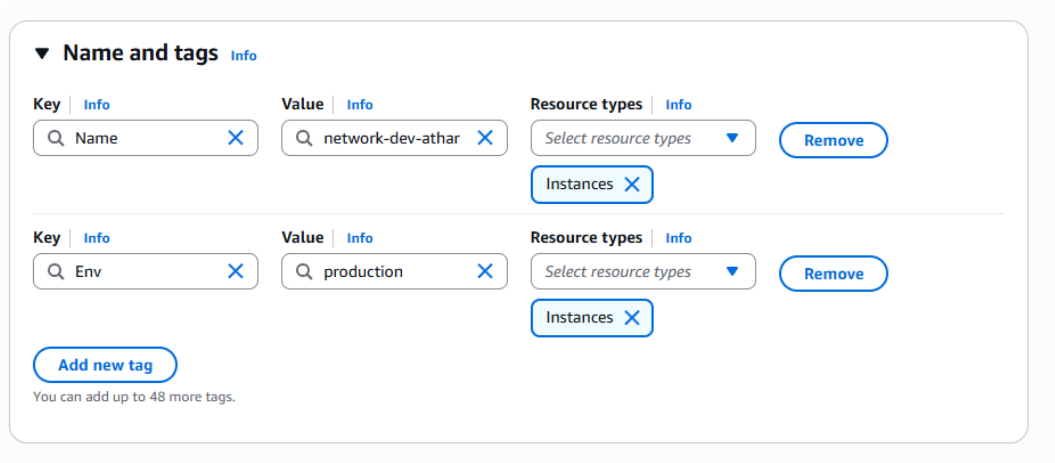

Figure 3: Production Instance Tagged with Env=production

Step 3: Created an IAM policy in JSON format to allow EC2 actions only on development instances.


```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "ec2:*",
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "ec2:ResourceTag/Env": "develc
        }
      }
    },
    {
      "Effect": "Allow",
      "Action": "ec2:Describe*",
      "Resource": "*"
    },
    {
      "Effect": "Deny",
      "Action": [
        "ec2:DeleteTags",
        "ec2:CreateTags"
      ],
      "Resource": "*"
    }
  ]
}
```

Figure 4: IAM Policy JSON (NextWorkDevEnvironmentPolicy)

Step 4: Created IAM user and group, and assigned the policy.

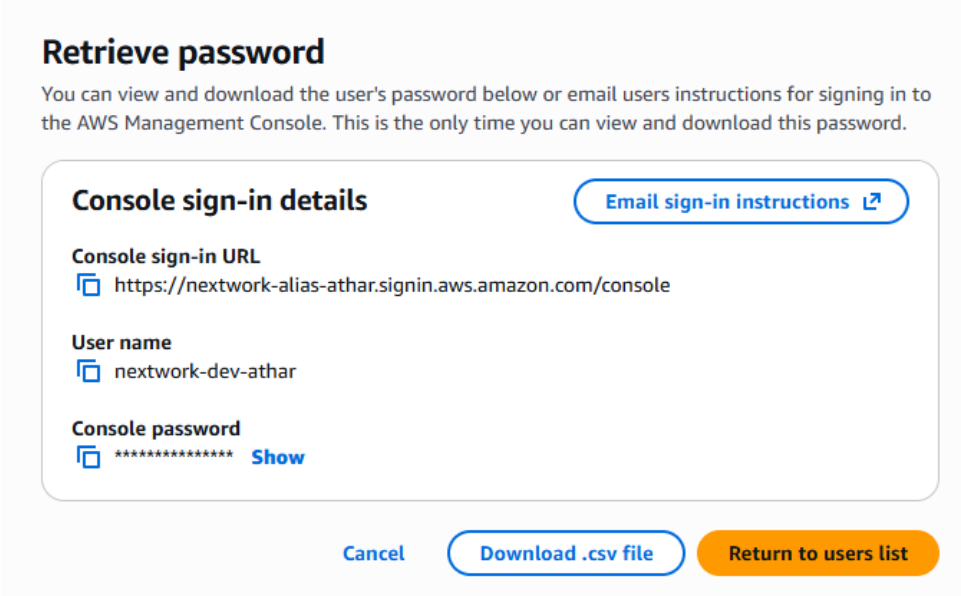

Figure 5: IAM User Creation for nextwork-dev-athar

5. Results

When testing user permissions, the IAM user was able to successfully perform operations on the **development instance** but was **denied access to the production instance**, confirming that the IAM policy functioned exactly as intended.

During validation and testing, the following key observations and outcomes were recorded:

- The **IAM user** belonging to the nextwork-dev-group could **start, stop, and describe** the EC2 instance tagged with Env=development.
- The same IAM user received an **AccessDenied error** when attempting to stop or modify the production instance (Env=production), proving that the **tag-based condition** ("ec2:ResourceTag/Env": "development") in the IAM policy was enforced.
- The **deny rule** explicitly blocking tag creation and deletion (ec2:CreateTags, ec2:DeleteTags) prevented the user from tampering with resource tags, ensuring the **integrity of environment boundaries**.
- The **IAM Policy Simulator** results matched the console tests, confirming that **policy evaluation logic** in AWS IAM processed both *Allow* and *Deny* statements correctly.
- The **development user** could still use ec2:Describe* actions to list all EC2 resources without being able to modify them, allowing visibility without control — a best practice for read-only access.

- This test validated the **Principle of Least Privilege (PoLP)** — each user could perform only the actions necessary for their role.
- The **separation of environments** ensured that developers could work safely without impacting live (production) infrastructure.
- This configuration reflected a **real-world enterprise setup**, where IAM policies enforce access boundaries using resource metadata instead of static resource lists.
- Overall, the results confirmed that the **IAM policy design was both functional and secure**, balancing flexibility for development and protection for production workloads.
- The environment could be extended to multiple users or teams by reusing the same tag-based IAM approach, making it **scalable, maintainable, and compliance-friendly**.

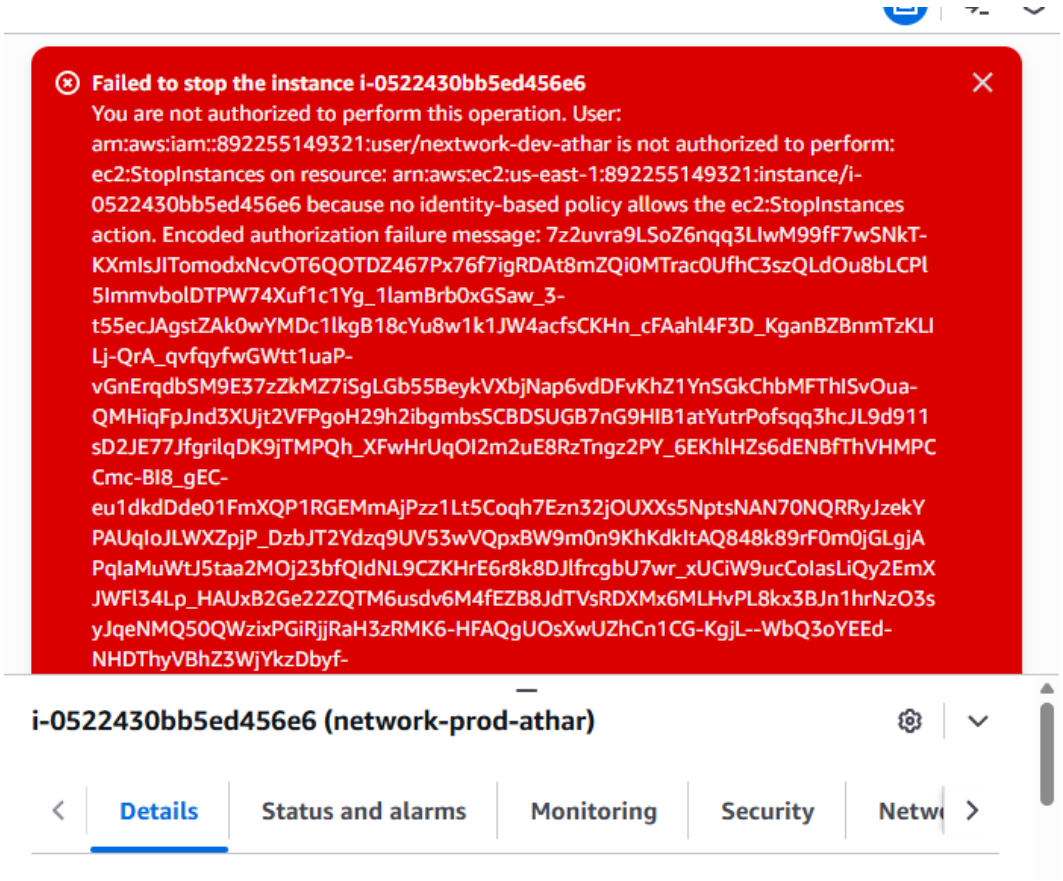

Figure 6: Access Denied – Attempt to Stop Production Instance

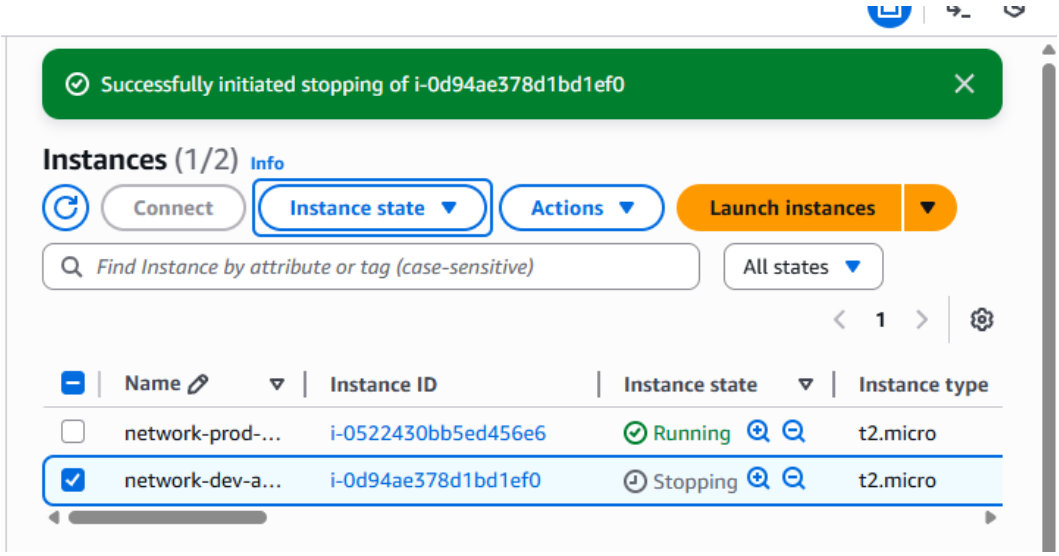

Figure 7: Successful Stop of Development Instance

6. Security Best Practices Applied

- Principle of Least Privilege – users only have access to what they need.
- Tag-based access control for environment-specific permissions.
- Group-based policy assignment for efficient management.
- Restricted tag manipulation to preserve resource integrity.
- Attribute-Based Access Control (ABAC) using resource tags.
- Separation of duties between admin and development roles
- Environment isolation between production and development.
- Policy validation using IAM Policy Simulator before deployment.
- Auditability through CloudTrail for access tracking.
- Alignment with AWS Well-Architected and security best practices.

7. Challenges Faced & Solutions

- Faced syntax issues with tag-based IAM conditions → fixed using `"ec2:ResourceTag/Env"`.
- Initial over-permissive access → resolved by adding **explicit deny** rules.
- Policy testing confusion → verified using **IAM Policy Simulator**.
- Manual user setup inefficiency → improved with **group-based permissions**.
- Ensured environment isolation through strict **tag-based access**.

8. Conclusion

The **AWS IAM and EC2 Access Control Project** successfully demonstrates the secure and structured implementation of **role-based access control (RBAC)** in a modern cloud environment. Through the combined use of **IAM policies**, **environment tagging**, and **user group management**, we effectively enforced a separation between production and development resources—ensuring that only authorized users could interact with designated environments.

This project showcased the power of **tag-based conditional access control** using AWS IAM's Condition element, where access permissions were dynamically applied based on resource tags. The result was a flexible, scalable model that allows organizations to manage permissions across multiple environments without the need for repetitive or static policy definitions. The integration of tag-based control also enhanced operational efficiency, reduced administrative overhead, and strengthened adherence to the **Principle of Least Privilege (PoLP)** — a foundational cybersecurity concept.

Additionally, by denying tag modification actions and restricting production access, the project emphasized **environment integrity** and **change control discipline**, two critical elements of cloud security governance. Testing with both the **IAM Policy Simulator** and the **AWS Management Console** validated that the policies behaved as intended, ensuring reliable and predictable security enforcement.

From a cybersecurity perspective, this project not only met its technical objectives but also reflected **real-world DevSecOps principles**. It demonstrated how cloud teams can maintain agility in development workflows while protecting production assets through policy-driven access management. The approach aligns with enterprise security frameworks such as **NIST SP 800-53**, **ISO 27001**, and the **AWS Well-Architected Framework's Security Pillar**, ensuring compliance readiness and operational resilience.

In conclusion, this project reinforced a deep understanding of **cloud security architecture**, **identity governance**, and **policy-driven access control**. It serves as a practical foundation for future exploration of advanced AWS security features such as **multi-factor authentication (MFA)**, **service-linked roles**, and **automated compliance monitoring**. Ultimately, it bridges the gap between theoretical cybersecurity concepts and their **real-world application in secure cloud infrastructure management**.